

Real-Time Personal Protective Equipment (PPE) Compliance Monitoring System

Final Project Report

Submitted in partial fulfillment of the requirements for the
Certification in Artificial Intelligence and Machine Learning

Muslim Uddin

Indian Institute of Technology (IIT) Guwahati

March 6, 2026

Abstract

Workplace safety in industrial and construction environments is heavily reliant on the strict adherence to Personal Protective Equipment (PPE) protocols. Traditional manual monitoring is prone to human error, fatigue, and scalability limitations. This project introduces a robust, real-time PPE Compliance Monitoring System designed to autonomously detect and verify the proper usage of safety helmets and high-visibility vests. Leveraging a custom-trained YOLOv8n (You Only Look Once version 8 nano) model, the system identifies three distinct classes: *person*, *helmet*, and *vest*. To determine compliance, a novel spatial filtering algorithm was developed. This algorithm computes the geometric center of detected PPE items and evaluates their spatial intersection with the bounding box of a detected person, successfully classifying the individual as "SAFE" or "UNSAFE." The architectural backbone consists of a highly optimized Node.js and Express backend interfaced with a React.js frontend dashboard. To achieve zero-lag, high-FPS webcam inference, the system employs a dual-processing approach: REST APIs manage static media, while a continuously running Python Daemon processes live video streams via WebSockets, eliminating model reload overhead. Empirical evaluation demonstrates highly effective performance, achieving a mean Average Precision (mAP@0.5) of 0.884 and processing live feeds at 45 Frames Per Second (FPS) on a standard consumer GPU. The resulting system is scalable, accurate, and ready for deployment in dynamic industrial environments.

Contents

Abstract	1
1 Introduction	3
1.1 Problem Statement	3
1.2 Objectives	3
1.3 Scope of the Project	3
2 Dataset Preparation	3
2.1 Data Collection and Curation	3
2.2 Annotation and Class Imbalance	4
2.3 Exporting to YOLO Format	4
3 Methodology & Training	4
3.1 YOLOv8 Architecture	4
3.2 Training Process	5
3.3 Algorithmic Logic: Spatial Filtering for Compliance	5
4 System Architecture & Integration	6
4.1 Frontend Architecture (React.js)	6
4.2 Backend Architecture (Node.js & Express)	6
4.3 The Python YOLO Daemon	6
5 Results & Evaluation	7
5.1 Model Metrics	7
5.2 Real-Time Performance	7
6 Conclusion & Future Work	7
6.1 Conclusion	7
6.2 Future Work	8

1 Introduction

1.1 Problem Statement

Occupational hazards in construction, manufacturing, and heavy industry result in thousands of preventable injuries and fatalities annually. A significant proportion of these incidents occur due to the non-compliance or improper usage of Personal Protective Equipment (PPE), specifically hard hats (helmets) and high-visibility safety vests. Manual surveillance of large worksites is not only labor-intensive but also inherently flawed due to human fatigue and blind spots. Consequently, there is an urgent need for an automated, intelligent system capable of monitoring PPE compliance in real-time, alerting safety officers to violations instantly.

1.2 Objectives

The primary objective of this project is to design, train, and deploy an end-to-end computer vision pipeline capable of detecting PPE non-compliance in real-time. The specific sub-objectives are as follows:

- Train a lightweight object detection model (YOLOv8n) to recognize *persons*, *helmets*, and *vests* with high accuracy under varying lighting and environmental conditions.
- Formulate a deterministic spatial logic algorithm to map detected PPE to the corresponding individuals.
- Develop a low-latency, full-stack application capable of streaming live video feeds, running inference without bottlenecking, and displaying results on an intuitive dashboard.

1.3 Scope of the Project

This project encompasses the full AI lifecycle: from dataset curation and data augmentation to model training, backend API development, and frontend integration. The current scope is limited to the detection of helmets and vests. The system is designed to process both static files (images and pre-recorded videos) and live web-camera streams.

2 Dataset Preparation

2.1 Data Collection and Curation

To ensure the model’s robustness, a diverse dataset was assembled by combining publicly available open-source industrial datasets (e.g., Roboflow Universe) with custom-scraped images of diverse construction sites. The aggregate dataset comprised 5,432

high-resolution images featuring workers in various postures, lighting conditions, and occlusion scenarios.

2.2 Annotation and Class Imbalance

The images were manually verified and annotated using standard bounding-box annotation tools. The labels were strictly mapped to three classes:

- **Class 0:** Person
- **Class 1:** Helmet
- **Class 2:** Vest

Initial exploratory data analysis revealed a mild class imbalance, with "Person" annotations outnumbering "Vest" annotations by a ratio of 3:1. To mitigate this, targeted data augmentation techniques were applied. Mosaic augmentation, horizontal flipping, random cropping, and color jitter (brightness, contrast, and saturation adjustments) were utilized to synthetically balance the class distribution and improve the model's generalization capabilities.

2.3 Exporting to YOLO Format

Following augmentation, the dataset was normalized and split into Training (80%), Validation (10%), and Testing (10%) subsets. The bounding box coordinates were normalized to relative coordinates (x_{center} , y_{center} , $width$, $height$) ranging from 0 to 1, conforming to the strict formatting requirements of the YOLO PyTorch training pipeline.

3 Methodology & Training

3.1 YOLOv8 Architecture

The YOLOv8n (nano) architecture was selected as the core AI model due to its optimal balance between parameter efficiency and Mean Average Precision (mAP). YOLOv8 introduces an anchor-free detection head, which reduces the number of box predictions and speeds up Non-Maximum Suppression (NMS). Its backbone utilizes a modified CSP-Darknet53 feature extractor coupled with a Path Aggregation Network (PANet) neck, ensuring rich spatial and semantic feature fusion critical for detecting small objects like helmets at a distance. The "nano" variant was specifically chosen to guarantee high FPS processing on edge devices or standard hardware without requiring heavy GPU clusters.

3.2 Training Process

The model was trained using the PyTorch framework. Transfer learning was employed by initializing the network with weights pre-trained on the COCO dataset. The training hyperparameters were empirically tuned as follows:

- **Epochs:** 100
- **Optimizer:** AdamW (Adaptive Moment Estimation with Weight Decay)
- **Learning Rate (Initial):** 0.001 with cosine annealing learning rate scheduler.
- **Batch Size:** 16
- **Image Resolution:** 640×640 pixels

The training process stabilized around epoch 85, showing convergence in both bounding box regression loss and classification loss, indicating that the model was neither overfitting nor underfitting.

3.3 Algorithmic Logic: Spatial Filtering for Compliance

Detecting a helmet and a person independently does not guarantee that the person is wearing the helmet. To establish ownership and compliance, a custom spatial filtering algorithm was developed.

Let a detected person’s bounding box be defined by its top-left and bottom-right coordinates as $P = (x_{p1}, y_{p1}, x_{p2}, y_{p2})$. Let a detected PPE item (helmet or vest) have bounding box coordinates $E = (x_{e1}, y_{e1}, x_{e2}, y_{e2})$.

First, the algorithm calculates the geometric center (C_x, C_y) of the PPE item:

$$C_x = \frac{x_{e1} + x_{e2}}{2}, \quad C_y = \frac{y_{e1} + y_{e2}}{2} \quad (1)$$

Next, the system checks if the center of the PPE item falls strictly within the boundaries of the person’s bounding box. The condition for spatial intersection is:

$$(x_{p1} \leq C_x \leq x_{p2}) \wedge (y_{p1} \leq C_y \leq y_{p2}) \quad (2)$$

If this condition holds true for both a helmet and a vest for a specific person bounding box, the person is classified as **SAFE**. If either the helmet or the vest is missing (i.e., their center points do not intersect with P), the person is flagged as **UNSAFE**. This logic is highly efficient ($O(N \times M)$ where N is persons and M is PPE items) and easily computes in real-time.

4 System Architecture & Integration

Creating a real-time web application for computer vision poses a significant challenge: loading a neural network model into memory for every frame introduces catastrophic latency. To solve this, a dual-processing, highly decoupled full-stack architecture was implemented.

4.1 Frontend Architecture (React.js)

The user interface is a responsive dashboard built with React.js. It features interactive visibility toggles allowing users to turn bounding boxes for specific classes on or off. Furthermore, it supports client-side image resizing to reduce bandwidth overhead before transmission. The UI includes historical scan tracking, storing past compliance violations, and a side-by-side comparison view displaying the raw feed next to the inferred feed.

4.2 Backend Architecture (Node.js & Express)

The backend server is powered by Node.js and Express. For static files (images and videos), standard REST APIs are utilized to handle uploads, trigger inference, and return the annotated media. However, for live webcam feeds, REST is insufficient due to HTTP overhead. Here, Socket.io is implemented to establish a persistent, bi-directional WebSocket connection with the React client.

4.3 The Python YOLO Daemon

To achieve zero-lag inference, the backend spawns a continuous background Python Daemon upon server startup.

1. The Python script loads the YOLOv8n PyTorch model into VRAM **only once**.
2. It then listens to an inter-process communication (IPC) channel or local socket.
3. The React frontend captures webcam frames and streams them as base64-encoded strings via Socket.io to Node.js.
4. Node.js pipes these frames directly into the running Python Daemon.
5. The Daemon performs inference, applies the spatial filtering logic, draws the bounding boxes, and returns the processed base64 frame back through the pipeline to the frontend.

This architecture completely bypasses the model-loading bottleneck, allowing the system to achieve fluid, real-time video processing.

5 Results & Evaluation

The custom-trained YOLOv8n model and the integrated web system were evaluated on both static test datasets and live webcam environments.

5.1 Model Metrics

The model exhibited excellent detection capabilities, particularly in distinguishing between workers and background machinery. The quantitative evaluation on the hold-out test set yielded the following metrics:

Table 1: Model Evaluation Metrics on Test Dataset

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Person	0.891	0.865	0.912	0.684
Helmet	0.843	0.802	0.865	0.582
Vest	0.822	0.763	0.875	0.597
Overall	0.852	0.810	0.884	0.621

5.2 Real-Time Performance

The system’s latency was tested to validate the Python Daemon WebSocket architecture. On a consumer-grade GPU (NVIDIA RTX 3060), the pipeline processed live 720p webcam streams at an average of **45 FPS**. When tested purely on a standard multi-core CPU (Intel i7), the system maintained a respectable **15 to 18 FPS**, proving that the ”nano” architecture and IPC pipeline are highly optimized and suitable for deployment on edge devices without dedicated hardware accelerators.

6 Conclusion & Future Work

6.1 Conclusion

This project successfully designed and implemented a Real-Time PPE Compliance Monitoring System. By custom-training a YOLOv8n model and integrating it with a robust mathematical spatial filtering algorithm, the system accurately identifies safety violations with an overall mAP of 0.884. Furthermore, the innovative architectural decision to bridge a React/Node.js stack with a continuous Python Daemon via WebSockets eliminated inference bottlenecks, resulting in a fluid 45 FPS live stream experience. This system proves the viability of using edge-AI for scalable, automated occupational safety monitoring.

6.2 Future Work

While the current system is highly functional, future iterations could expand on the following areas:

- **Expanded Class Detection:** Integrating additional PPE classes such as safety goggles, industrial work boots, and safety gloves.
- **Object Tracking:** Implementing tracking algorithms like DeepSORT to track persistent violators across multiple camera feeds and prevent duplicate logging of the same violation.
- **Edge Deployment:** Converting the PyTorch weights to TensorRT or ONNX formats to further accelerate inference on embedded systems like NVIDIA Jetson Nano for localized, off-the-grid deployment at remote construction sites.